

PythonIntermediateCourse-es009

June 3, 2026

1 Curso básico de Python

1.1 Apuntes intermedio

Curso básico de Python. Apuntes intermedio por Marcelo Horacio Fortino. Versión 2.5.1. Junio 2026.

Esta obra está sujeta a la licencia Reconocimiento-CompartirIgual 4.0 Internacional de Creative Commons. Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-sa/4.0/>. Puede hallar permisos más allá de los concedidos con esta licencia en <https://fortinux.com>. Sugerencias y comentarios a info@fortinux.com.

Todas las marcas son propiedad de sus respectivos dueños. Python® y PyCon® son marcas registradas de la Python Software Foundation. Linux® es una marca registrada de Linus Torvalds. Ubuntu® es una marca registrada de Canonical Limited. Google® es una marca registrada de Google Inc. Microsoft® y Windows® son marcas registradas de Microsoft Corporation. Red Hat® es una marca comercial registradas de Red Hat, Inc. UNIX® es una marca comercial registrada de The Open Group. Apple® y Mac OS® son marcas comerciales de Apple Inc.

Versión	Autor/es	Fecha	Observaciones
1.0	Marcelo Horacio Fortino	2021/Marzo	Curso Python
1.1	Marcelo Horacio Fortino	2021/Junio	Convertido a markdown - ipynb
1.2	Marcelo Horacio Fortino	2021/Agosto	Actualizados contenidos
1.3	Marcelo Horacio Fortino	2021/Octubre	Agregado Flask microframework
1.4	Marcelo Horacio Fortino	2021/Noviembre	Agregado Pandas - datascience
1.5	Marcelo Horacio Fortino	2021/Diciembre	Agregado Devops - Ansible
2.0	Marcelo Horacio Fortino	2022/Abril	Nueva estructura: core / module
2.1	Marcelo Horacio Fortino	2022/Junio	Módulo apuntes intermedio
2.2	Marcelo Horacio Fortino	2022/Agosto	Actualizado temario y ejercicios

Versión	Autor/es	Fecha	Observaciones
2.3	Marcelo Horacio Fortino	2022/Octubre	Actualizado despliegue a render.com
2.4	Marcelo Horacio Fortino	2023/Noviembre	Aplicaciones de escritorio con python
2.5	Marcelo Horacio Fortino	2026/Abril	Aplicaciones de ML e IA para python

Esta obra se distribuye con la esperanza de que sea útil, pero SIN NINGUNA GARANTÍA, incluso sin la garantía MERCANTIL implícita o sin garantizar la CONVENIENCIA PARA UN PROPÓSITO PARTICULAR. El autor no asume ninguna responsabilidad si el lector hace un mal uso de la misma.

Estos apuntes se basan en: - La documentación oficial de Python, <https://docs.python.org/es/3/tutorial/index.html>, - La bibliografía presentada al final de este documento, y - Documentación propia recogida a lo largo de los años de diversas fuentes.

1.2 Objetivo del curso

- Objetivo general del curso:
- Aprender a usar Python para crear scripts y programas simples plenamente funcionales, desarrollar aplicaciones web, realizar análisis de datos, y utilizar bibliotecas de inteligencia artificial (AI) para aprendizaje automático (ML).
- Objetivos específicos:
- Reconocer las características principales de Python y su utilidad práctica.
- Identificar tipos de datos (simples y compuestos) y operadores.
- Aplicar variables y estructuras de control de flujo.
- Construir funciones y clases (POO).
- Clasificar los módulos y paquetes por sus funcionalidades y objetivos.
- Comparar el lenguaje con otros similares de scripts, procedimentales y orientados a objetos.
- Probar bibliotecas para conexiones REST a aplicaciones web y bases de datos.
- Resolver problemas y errores en el código fuente proponiendo soluciones alternativas (refactoring).
- Utilizar la biblioteca pandas junto con matplotlib y numpy para realizar análisis estadísticos de datos y gráficos.
- Descubrir las herramientas en python de ML y DL (*Deep Learning*) para la inteligencia artificial.
- Como resultado práctico al final del curso cada estudiante habrá creado una aplicación web utilizando el microframework de Python Flask.

1.3 Temario

- Introducción, instalación y compilación
- Datos, expresiones y sentencias
- Variables y funciones, control de flujo

- Clases y objetos, herencia, polimorfismo
- Entradas y salidas con Python
- Gestión de módulos, paquetes y bibliotecas
- Servicios y programas en red, REST API
- Desarrollo de aplicaciones web con Flask
- Análisis de datos con pandas, matplotlib y numpy
- Aprendizaje automático (ML) e inteligencia artificial (AI) con Python
- Módulos opcionales
 - DevOps con Ansible
 - Plotting con Python
 - SDK de GCP para Python
 - SQL ejemplos con pandas
 - Docker y kubernetes con Python
 - Aplicaciones de escritorio con python

1.4 Bibliografía

- Downey, A., Elkner, J., Meyers, C. (2015). Aprenda a Pensar Como un Programador con Python.
Recuperado de <https://argentinaenpython.com/quiero-aprender-python/aprenda-a-pensar-como-un-programador-con-python.pdf>
- Kent D. Lee. (2014). Python, Programming Fundamentals Second Edition.
- Marzal Varó, A., Gracia Luengo, I., García Sevilla, Pedro. (2014). Introducción a la programación con Python 3.
Recuperado de <http://repositori.uji.es/xmlui/handle/10234/102653>
- Miller, B., Ranum, D. Solución de problemas con algoritmos y estructuras de datos usando Python. Traducido por Mauricio Orozco-Alzate, Universidad Nacional de Colombia - Sede Manizales.
Recuperado de <https://runestone.academy/ns/books/published/pythoned/index.html?mode=browsing>
- Shaw, Z. A. (2016). Learn Python 3 the Hard Way.
Recuperado de <https://learnpythonthehardway.org/>
- Van Rossum, G. and the Python development team. (2020). Documentación de Python en español.
Recuperado de https://python-docs-es.readthedocs.io/_/downloads/es/pdf/pdf/
- McGregor, S. E. (2022). Practical Python data wrangling and data quality. O'Reilly Media.
- Chollet, F. (2017). Deep learning with Python. Manning Publications.
- Russell, S., & Norvig, P. (2019). Artificial intelligence: A modern approach. Pearson Higher Education.
- D2L.ai: Interactive Deep Learning Book with Multi-Framework Code, Math, and Discussions.
Recuperado de <https://d2l.ai/>

2 Análisis de datos con python

- Introducción, La biblioteca pandas, Accediendo al DataFrame, Indexado en pandas, Filtros, Ordenar datos, Actualización de datos, Métodos para cambiar datos, Eliminar datos, Agregar y/o eliminar columnas y filas, Gráficos en pandas, Agrupar datos, Agregar datos, Exportar datos.

2.1 Introducción

- A partir de la versión 3.4 python cuenta con un módulo para calcular estadísticas matemáticas de datos numéricos (de tipo Real).
- Es un módulo con funciones básicas a nivel de calculadora científica, por lo tanto no reemplaza a las bibliotecas NumPy o SciPy, ni al software propietario profesional como Minitab, SAS o Matlab.
- Entre las funciones estadísticas disponibles, se pueden mencionar las medias aritmética, geométrica, harmónica, y la mediana; entre otras.
- Fuente: <https://docs.python.org/es/3/library/statistics.html>.
- Algunos paquetes útiles para análisis de datos:
 - Pandas <https://pandas.pydata.org/>.
 - Numpy <https://numpy.org/>.
 - Scipy <https://scipy.org/>.
 - Polars <https://pola.rs/>.
- Visualización de datos
 - Dash <https://pypi.org/project/dash/>: Utiliza Plotly.js <https://github.com/plotly/plotly.js>, React <https://reactjs.org/> y Flask <https://palletsprojects.com/p/flask/> como base.
 - Plotly <https://plotly.com>.
 - Seaborn <https://seaborn.pydata.org/> basada en Matplotlib <https://matplotlib.org/>.
 - Vega-Altair <https://altair-viz.github.io/>.

3 La biblioteca pandas

- Los ejemplos desarrollados en estos apuntes han tomado como guía a la serie *Python Pandas Tutorial* de Corey Schafer
- <https://www.youtube.com/watch?v=ZyhVh-qRZPA&list=PL-osiE80TeTsWmV9i9c58mdDCSskIFdDS>
- Para realizar esta práctica se deberá tener python y el módulo jupyter instalado en la máquina local.
- Otra opción es abrir una cuenta gratuita en Colaboratory:
- <https://colab.research.google.com/>
- *Pandas es una herramienta de análisis y manipulación de datos de código abierto rápida, potente, flexible y fácil de usar, construida sobre el lenguaje de programación Python.*
- Fuente: <https://pandas.pydata.org/>.

- Bibliografía: McKinney, Wes and the Pandas Development Team. pandas: powerful Python data analysis toolkit. Release 1.4.2. (2022).
- Recuperado de <https://pandas.pydata.org/docs/pandas.pdf>.
- Documentación recomendada para comenzar a trabajar con pandas:
- La guía oficial: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html.
- 10 minutes to Pandas: https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html.
- Cookbook: https://pandas.pydata.org/pandas-docs/stable/user_guide/cookbook.html.
- Pandas Cheat Sheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf.

```
[ ]: # Obtener de Internet y descomprimir ficheros utilizando Python
from io import BytesIO
from urllib.request import urlopen
from zipfile import ZipFile
```

```
[ ]: # Obtener el fichero con los datos de https://insights.stackoverflow.com/survey
# Pertenecen a la encuesta anual de desarrolladores año 2023
zip_url = 'https://cdn.stackoverflow.co/files/jo7n4k8s/production/
49915bfd46d0902c3564fd9a06b509d08a20488c.zip/
↳stack-overflow-developer-survey-2023.zip'
```

```
[ ]: # Mostrar el directorio actual
pwd
```

```
[ ]: # Extraer el fichero zip en el directorio actual
with urlopen(zip_url) as zipresp:
    with ZipFile(BytesIO(zipresp.read())) as zfile:
        zfile.extractall('/home/usuario/')
```

```
[ ]: pip install --upgrade pip
```

```
[ ]: pip install pandas
```

```
[ ]: import pandas as pd
```

```
[ ]: pwd
```

- Los dos componentes principales de pandas son las *Series* y el *DataFrame*.
- Una *Serie* es básicamente una columna.
- Un *DataFrame* es una tabla multi-dimensional compuesta de *Series*.

```
[ ]: # Se crea el dataframe
datos = pd.read_csv('survey_results_public.csv')
```

```
[ ]: # shape muestra la cantidad de filas y columnas del fichero en forma de tupla
datos.shape

[ ]: # Muestra la información sobre el dataframe
datos.info()

[ ]: # Muestra todas las columnas del fichero
pd.set_option('display.max_columns', 48)
pd.set_option('display.max_rows', 50)

[ ]: # Se crea el schema del dataframe
schema_datos = pd.read_csv('survey_results_schema.csv')

[ ]: schema_datos

[ ]: # Muestra los 5 primeros registros del dataframe
datos.head()

[ ]: # Muestra los últimos 10 registros
datos.tail(10)
```

3.1 Accediendo al DataFrame

- Las *Series* (columnas) son listas de filas de datos (*one dimensional arrays*).
- Los *DataFrames* son matrices (*two dimensional arrays*) de series de dos dimensiones (filas y columnas).

```
[ ]: # Accede a los datos de la columna 'Employment'
datos.Employment

[ ]: # Con una sola columna, el par de corchetes (brackets) devuelve una Serie
datos['Employment']

[ ]: # Con corchetes dobles devuelve el dataframe
datos[['Employment']]

[ ]: # Accede a los datos de las columnas 'Employment', 'Country', 'YearsCode'
datos[['Employment', 'Country', 'YearsCode']]

[ ]: # Muestra las columnas del dataframe
datos.columns
```

- *iloc* realiza la indexación puramente basada en la ubicación de enteros para la selección por posición.
- Fuente: <https://pandas.pydata.org/docs/reference/api/pandas.Series.iloc.html>.
- Básicamente significa que accede a los elementos de la lista mediante la posición del índice.

```
[ ]: # Accede a filas con iloc (integer location)
datos.iloc[0]
```

```
[ ]: datos.iloc[[0, 3, 5]]
```

```
[ ]: # Muestra la columna 3 de las filas 0, 3, y 5
datos.iloc[[0, 3, 5], 18]
```

- *loc* en cambio accede a un grupo de filas y columnas por etiqueta(s) o una matriz booleana.
- Fuente: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.loc.html>.

```
[ ]: # loc muestra resultados buscando por etiquetas (labels)
datos.loc[[0, 3, 5], 'Country']
```

```
[ ]: # Cuenta los países de donde provienen los programadores
datos.Country.value_counts()
```

3.2 Indexado en pandas

```
[ ]: # Determinar una columna como identificador primario
datos.set_index('ResponseId')
# Para establecerla
# datos.set_index('ResponseId', inplace=True)
# Para volver al ID original
# datos.reset_index(inplace=True)
```

```
[ ]: datos
```

```
[ ]: # Determinar una columna como identificador primario al crear el dataframe
schema_datos = pd.read_csv('survey_results_schema.csv', index_col='qname')
```

```
[ ]: schema_datos
```

```
[ ]: # Se accede al dato a través del campo 'Country'
schema_datos.loc['Country', 'question']
```

```
[ ]: # Ordenar los datos: ascendente
schema_datos.sort_index()
# Ordenar los datos: descendente
# schema_datos.sort_index(ascending=False)
```

3.3 Filtros

```
[ ]: # Muestra todos los registros que cumplen con el criterio
filtro = datos['Country'] == 'Spain'

[ ]: datos[filtro]

[ ]: # Datos filtrados usando loc
datos.loc[filtro, 'Employment']

[ ]: prog_python = (datos['Country'] == 'Spain') & (datos['LanguageHaveWorkedWith']_
↳ == 'Python')

[ ]: datos.loc[prog_python]

[ ]: filtrar_paises = (datos['Country'] == 'Spain') | (datos['Country'] == 'France')

[ ]: datos.loc[filtrar_paises]
# Negación
# datos.loc[~filtrar_paises]

[ ]: salario_alto = datos['ConvertedCompYearly'] > 80000

[ ]: datos.loc[salario_alto]

[ ]: datos.loc[salario_alto, ['Country', 'LanguageHaveWorkedWith']].head(10)

[ ]: salario_eur = (datos['ConvertedCompYearly'] > 80000) & (datos['Currency'] ==_
↳ 'EUR European Euro')

[ ]: datos.loc[salario_eur]

[ ]: datos['LanguageHaveWorkedWith']

    • Verifica si un patrón está contenido en una cadena. Se puede utilizar para NaN.
    • Fuente https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.contains.html

[ ]: filtrar_lenguajes = datos['LanguageHaveWorkedWith'].str.contains('Python',_
↳ na=False)

[ ]: filtrar_lenguajes

[ ]: datos.loc[filtrar_lenguajes, 'LanguageHaveWorkedWith']
```


3.4 Ordenar datos

```
[ ]: datos.sort_values(by="Country")

[ ]: datos.sort_values(by="Country", ascending=False)

[ ]: datos.sort_values(by=["Country", "ConvertedCompYearly"])

[ ]: # COUNTRY ascendente, ConvertedCompYearly descendente
datos.sort_values(by=["Country", "ConvertedCompYearly"], ascending=[True, ↵
↪False])

[ ]: # Modifica datos
# datos.sort_values(by=["Country", "ConvertedCompYearly"], ascending=[True, ↵
↪False], inplace=True)
# Reestablece el orden original
# datos.sort_index()

[ ]: # Ordena una columna
datos["Country"].sort_values()

[ ]: datos["Country"].head(20)

[ ]: datos.sort_values(by=["ConvertedCompYearly", "Country"], ascending=[False, ↵
↪True]).head(50)

[ ]: datos["ConvertedCompYearly"].nlargest(25)

[ ]: datos.nlargest(25, "ConvertedCompYearly")

[ ]: datos.nsmallest(25, "ConvertedCompYearly")
```

3.5 Actualización de datos

```
[ ]: datos.columns

[ ]: # Cambiar a mayúsculas el campo country
datos['Country'] = datos['Country'].str.upper()

[63]: # Cambia a mayúsculas las etiquetas de los campos
datos.columns = (x.upper() for x in datos.columns)

[ ]: datos

[ ]: # Reemplazar caracteres en las etiquetas de los campos
datos.columns = datos.columns.str.replace('_', '')
datos
```

```
[ ]: # Cambiar el nombre a un campo
datos.rename(columns={'RESPONSEID': 'ID'})
# Para establecerlo
# datos.rename(columns={'RESPONSEID': 'ID'}, inplace=True)
```

```
[ ]: datos.loc[1]
```

```
[68]: # Cambiar un dato a un registro
datos.loc[1, ['COUNTRY', 'LANGUAGEWANTTOWORKWITH']] = ['Spain', 'Python']
```

```
[ ]: datos.loc[[1]]
```

```
[73]: datos.loc[1, 'COUNTRY'] = 'Netherlands'
datos.at[1, 'COUNTRY'] = 'Netherlands' # Idéntico resultado a datos.loc
```

```
[74]: filtro_modificar_pais = (datos['COUNTRY'] == 'Spain')
datos.loc[filtro_modificar_pais, 'COUNTRY'] = 'España'
```

```
[ ]: datos[filtrar_paises]
```

```
[ ]: datos
```

3.6 Métodos para cambiar datos

- Existen 4 métodos para cambiar datos
 - apply: llama a una función con un dataframe o serie de objetos
 - map: es más rápido que *replace*
 - applymap: solo funciona en dataframes
 - replace

```
[ ]: # Ejemplos de apply en series de objetos
# Accede a un campo y devuelve las columnas y filas
datos['COUNTRY'].apply(len)
```

```
[85]: def actualizar_pais(COUNTRY):
      return COUNTRY.lower()
```

```
[ ]: datos['COUNTRY'].apply(actualizar_pais)
```

```
[ ]: datos['COUNTRY'] = datos['COUNTRY'].apply(lambda z: z.upper())
```

```
[ ]: datos
```

```
[ ]: # Ejemplos de apply en dataframes
# Ejecuta el código en cada serie del dataframe
datos.apply(len)
```

```
[ ]: # Ejecuta el código en cada serie del dataframe
# modificando el eje a columna
datos.apply(len, axis='columns')
```

```
[ ]: # Ejemplos de apply en series
# Ejecuta el código en cada cada valor de la serie
datos.apply(pd.Series)
```

```
[ ]: # Ejemplos de applymap en dataframes
# Ejecuta el código (función) en cada serie del dataframe
datos.applymap
```

```
[ ]: # El método map solamente funciona con series
# Permite cambiar los valores de las series
datos['SOACCOUNT']
```

```
[ ]: datos['SOACCOUNT'].map({'Yes': True, 'No': False})
```

```
[94]: # Aplicar los cambios
datos['SOACCOUNT'] = datos['SOACCOUNT'].map({'Yes': True, 'No': False})
```

```
[96]: # El método replace permite cambiar los valores de las series
# Se aplican los cambios creando una variable que utilice
# este código como en el ejemplo anterior
datos['COUNTRY'].replace({'Netherlands': 'PAISES BAJOS'})
```

```
[96]: 0          NaN
1      PAISES BAJOS
2      UNITED STATES OF AMERICA
3      UNITED STATES OF AMERICA
4      PHILIPPINES
...
89179      BRAZIL
89180      ROMANIA
89181      ISRAEL
89182      SWITZERLAND
89183      IRAN, ISLAMIC REPUBLIC OF...
Name: COUNTRY, Length: 89184, dtype: object
```

3.7 Eliminar datos

```
[ ]: # Mostrar celdas con NaN
datos.isna()
```

```
[ ]: # Obtener el porcentaje de celdas con NaN
datos.isna().mean()
```

```
[ ]: with pd.option_context('display.max_rows', None):  
      print(datos.isna().mean() < .7)
```

```
[ ]: # Eliminar columnas  
datos.drop(['AIDEVHAVEWORKEDWITH', 'AIDEVWANTTOWORKWITH'], axis = 1)
```

```
[ ]: # Ejemplos  
# datos.drop(labels=['indice_fila'], axis= 0, inplace=True)  
# datos.drop(labels=['nombre_columna'], axis= 1, inplace=False)
```

```
[ ]: # Elimina columnas con NaN  
# Para cambiar el DataFrame original se usa el argumento inplace = True  
# datos.dropna(inplace = True)
```

3.8 Agregar y/o eliminar columnas y filas

- Para transponer los datos de un dataframe:

```
[ ]: datos.head().T
```

```
[ ]: # Haciendo un join de dos columnas con un espacio para delimitarlas  
datos['SOVISITFREQ'] + ' ' + datos['NEWSOSITES']
```

```
[107]: # Crea la nueva columna AYUDA  
datos['AYUDA'] = datos['SOVISITFREQ'] + ' ' + datos['NEWSOSITES']
```

```
[ ]: # Eliminar columnas  
# Otra opción: del datos['nombre_columna']  
# Aplicar los cambios con: inplace=True  
datos.drop(columns=['SOVISITFREQ', 'NEWSOSITES'])  
datos.columns
```

```
[ ]: # Dividir una columna  
# datos[['datos1', 'datos2']] = datos['nombre_columna'].str.split(' ',  
    ↪ expand=True)
```

```
[ ]: # Agregar una fila o registro: append() será eliminado en el futuro  
# datos.append({'LANGUAGEWANTTOWORKWITH': 'Python'}, ignore_index=True)
```

```
[112]: # Agregar una fila o registro  
nuevo_registro = pd.DataFrame({'LANGUAGEWANTTOWORKWITH': 'Python'},  
    ↪ index=[89186])  
datos2 = pd.concat([nuevo_registro, datos.loc[:]]).reset_index(drop=True)
```

```
[113]: # Agregar otro dataframe: append() será eliminado en el futuro  
datos_doble = datos.append(datos)
```

```
/tmp/ipykernel_19512/2046567915.py:2: FutureWarning: The frame.append method is deprecated and will be removed from pandas in a future version. Use pandas.concat instead.
```

```
datos_doble = datos.append(datos)
```

```
[115]: # Agregar otro dataframe
datos_doble = pd.concat([datos, datos2])
```

```
[ ]: datos_doble
```

```
[117]: datos_doble = datos_doble.drop_duplicates()
```

```
[118]: datos_doble.shape
```

```
[118]: (89185, 85)
```

```
[ ]: # Eliminar una fila o registro
datos.drop(index=89183)
```

3.9 Gráficos en pandas

- Los métodos *plot*. se pueden aplicar a las *Series* y a los *DataFrames*.
- De forma predeterminada cada columna se grafica como un elemento distinto (area, bar, etc.)
- Cada gráfico creado con pandas es de tipo objeto *Matplotlib*.
- Para crear gráficos es necesario importar las bibliotecas *numpy*, *pandas* y *matplotlib*.

```
[123]: import numpy as np
import pandas as pd
```

- *Matplotlib* es una biblioteca completa para crear visualizaciones estáticas, animadas e interactivas en Python.
- Fuente: <https://matplotlib.org/>.

```
[ ]: pip install matplotlib
```

```
[125]: import matplotlib.pyplot as plt
```

```
[ ]: # Diagrama de caja de todas las columnas con datos numéricos
boxplot = datos.boxplot(figsize = (5,5))
```

```
[ ]: # Diagrama de caja de una columna
datos.boxplot(column = ['CONVERTEDCOMPYEARLY'], grid = False)
```

```
[ ]: # Elimina todos los valores superiores a 300000 U$S anuales para el salario
datos.drop(datos[datos['CONVERTEDCOMPYEARLY'] > 300000.0].index)
```

```
[ ]: # Elimina todos los valores inferiores a 100 U$S anuales para el salario
datos.drop(datos[datos['CONVERTEDCOMPYEARLY'] < 1000].index, inplace = True)

[ ]: datos.drop(datos[datos['CONVERTEDCOMPYEARLY'] > 200000].index, inplace = True)

[ ]: # diagrama de caja con los cambios
datos.boxplot(column = ['CONVERTEDCOMPYEARLY'], grid = False)

[ ]: # Histograma con los mismos datos
histograma = datos.hist(column='CONVERTEDCOMPYEARLY', bins=10, grid=True,
                        figsize=(12,8), color='#55bf91', zorder=2, rwidth=0.9)

[ ]: # Creamos la variable grafico
# datos.COUNTRY.value_counts(dropna=False). dropna no cuenta los NaN
grafico = (datos.COUNTRY.value_counts())

[ ]: grafico.plot()

[ ]: datos["CONVERTEDCOMPYEARLY"].plot()

[ ]: # Seleccionamos los primeros diez valores
grafico_cant_prog = (grafico.nlargest(10))

[ ]: # Utilizamos título, etiquetas y colores
grafico_cant_prog.plot.bar(title='Cantidad de programadores por país',
    ↪xlabel='Países',
                                ylabel='# de programadores', color=['red', 'green',
    ↪'blue'])

[ ]: # Utilizamos una paleta de colores
grafico_cant_prog.plot.bar(title='Cantidad de programadores por país',
                        xlabel='Países', ylabel='# de programadores',
    ↪colormap='plasma')

[ ]: # Métodos disponibles en Pandas para gráficos
[
    nombre
    for nombre in dir(datos.plot)
    if not nombre.startswith("_")
]

[ ]: datos["CONVERTEDCOMPYEARLY"].plot.box(xlabel="Salario anual", ylabel="Miles de
    ↪euros")
```

3.10 Agrupar datos

```
[ ]: datos.describe()

[ ]: datos["CONVERTEDCOMPYEARLY"].mean()

[ ]: datos.median()

[ ]: datos["MAINBRANCH"].value_counts()

[ ]: # datos["NEWOTHERCOMMS"].value_counts()
      # normalize=True convierte a porcentaje los valores
      datos["NEWOTHERCOMMS"].value_counts(normalize=True)
```

3.11 Agregar datos

```
[ ]: pais = datos.groupby(["COUNTRY"])

[ ]: pais.get_group("ESPAÑA")

[ ]: pais["MAINBRANCH"].value_counts().head(20)

[ ]: pais["CONVERTEDCOMPYEARLY"].median().head(20)

[ ]: pais["CONVERTEDCOMPYEARLY"].agg(["median", "mean", "count"])
```

3.12 Exportar datos

```
[ ]: filtro = datos['Country'] == 'Spain'
      datos_españa = datos.loc[filtro]
      datos_españa.head()

[ ]: print(datos_españa)

[ ]: # exportar a .csv
      datos_españa.to_csv('datos_españa.csv')

[ ]: # exportar a Excel
      pip install openpyxl

[ ]: datos_españa.to_excel('datos_españa.xlsx')
```